Hazard Analysis
Software Engineering

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

Table 1: Revision History

Date	**Developer(s)**	**Change**
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **2**

4 Critical Assumptions **2**

5 Failure Mode and Effect Analysis **2**

6 Safety and Security Requirements **2**

7 Roadmap **3**

8 Appendix — Generative AI Use Disclosure **4**

8.1 AI Tools Used 4

8.2 How and Where Used 4

8.3 Verification of AI-Generated Content 4

9 Appendix — Reflection **5**

[You are free to modify this template. —SS]

1 Introduction

A hazard is a system condition or failure that could contribute to an undesirable outcome, resulting in losses like inaccurate accreditation assessments, poor deadline recommendations, a loss of course information, or sensitive data breaches.

The Course Deadline Management system is intended to support McMaster University's Computing and Software Department in reviewing Software Engineering course outlines for compliance with applicable university policies and Canadian Engineering Accreditation Board (CEAB) requirements. The system also analyzes course schedules to recommend assignment and midterm deadlines that reduce overlapping student workloads, while leaving final scheduling decisions to instructors.

This document identifies potential hazards associated with the Course Deadline Management system, examines their consequences, and establishes a basis for developing safety and security requirements to mitigate these risks.

2 Scope and Purpose of Hazard Analysis

The purpose of this hazard analysis is to identify potential failures in the Course Deadline Management system that could negatively affect accreditation compliance reviews, course scheduling, or the handling of course information. The analysis covers the processing of course outlines, evaluation against McMaster policies and CEAB requirements, deadline conflict detection, and the generation of scheduling recommendations. The scope is limited to Software Engineering courses within the CAS department and excludes first-year courses, courses outside the department, and the security of the hosting infrastructure itself.

Potential losses resulting from system hazards include:

- Accreditation non-compliance, resulting in unaddressed violations of McMaster policies or CEAB requirements.
- Increased student workload conflicts due to overlapping assignment and midterm deadlines, potentially contributing to academic stress and more MSAF requests.
- Incorrect scheduling decisions resulting from inaccurate or misleading recommendations.

- Loss or corruption of course information, resulting in inaccurate course outline data and scheduling issues.
- Unauthorized disclosure of confidential information, including the master course sheet, compliance findings, and tentative assessment dates.
- Wasted administrator and instructor time due to false compliance findings, incorrectly identified deadline conflicts, or inaccurate recommendations.
- Loss of trust in the system due to repeated inaccurate or unreliable results, potentially leading users to stop using the system and reducing its intended benefits.

3 System Boundaries and Components

[Dividing the system into components will help you brainstorm the hazards. You shouldn't do a full design of the components, just get a feel for the major ones. For projects that involve hardware, the components will typically include each individual piece of hardware. If your software will have a database, or an important library, these are also potential components. —SS]

4 Critical Assumptions

[These assumptions that are made about the software or system. You should minimize the number of assumptions that remove potential hazards. For instance, you could assume a part will never fail, but it is generally better to include this potential failure mode. —SS]

5 Failure Mode and Effect Analysis

[Include your FMEA table here. This is the most important part of this document. —SS] [The safety requirements in the table do not have to have the prefix SR. The most important thing is to show traceability to your SRS. You might trace to requirements you have already written, or you might need to add new requirements. —SS] [If no safety requirement can be devised, other mitigation strategies can be entered in the table, including strategies involving providing additional documentation, and/or test cases. —SS]

6 Safety and Security Requirements

[Newly discovered requirements. These should also be added to the SRS. (A rationale design process how and why to fake it.) —SS]

7 Roadmap

[Which safety requirements will be implemented as part of the capstone timeline? Which requirements will be implemented in the future? —SS]

8 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

8.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

8.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

8.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

9 Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?
4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?